Smart-Navigation-System

Optimized Pathfinding System Using Graph Data Structures and Dijkstra's Algorithm

This is Submitted By:

KARREM VENKATA RAJESH KUMAR REDDY
MAHESH BABU G
RANIT BEURIA
MANVEER SINGH
Tusnik Kumar Manna

Under the Guidance of
DEEPAK KUMAR

School of Computing & Artificial Intelligence
Lovely Professional University, Phagwara

DECLARATION

We hereby certify that the project titled **"Metro Route Finder: An Optimized Pathfinding System Using Graph Data Structures and Dijkstra's Algorithm"** is a genuine representation of our work, conducted as part of the Project requirement at Lovely Professional University, Phagwara. This work was carried out under the supervision of **ASSIATANT Professor Deepak Kumar** during the period of June to July 2026. All the data and insights presented in this project report are derived from our dedicated efforts and are entirely original.

Name of Student 1: Mahesh Babu
Registration No.: 12513870

Name of Student 2: Ranit Beuria
Registration No.: 12500340

Name of Student 3: Karrem Vekata Rajesh Kumar Reddy
Registration No.: 12529971

Name of Student 4: Manveer Singh
Registration No.: 12508877

Name of Student 5:Tusnik Kumar Manna
Registration No.: 12529242

CERTIFICATE

This is to certify that the declaration statement made by this group of students is correct to the best of my knowledge and belief. They have completed this Project under my guidance and supervision. The present work is the result of their original investigation, effort and study. No part of the work has ever been submitted for any other degree at any University. The Project is fit for the submission in Lovely Professional University, Phagwara.

Deepak Kumar.

Designation: Assistant Professor
School of Computing & Artificial Intelligence
Lovely Professional University,
Phagwara, Punjab

Date: 19/07/2026.

ABSTRACT

This project presents an optimized metro route-finding system for efficient urban commuter navigation using graph-based algorithmic techniques. The transportation network is represented as a weighted graph in which metro stations are vertices and direct station-to-station connections are edges. The proposed solution uses an adjacency-list representation, a station-name lookup structure, and Dijkstra's single-source shortest-path algorithm to calculate an optimal route between a selected origin and destination. The design emphasizes correctness, memory efficiency, modularity, clear error handling, and the ability to scale from a small academic network to a substantially larger metro topology.

The proposed framework incorporates Dijkstra's algorithm to compute the absolute shortest path while minimizing computational overhead. An adjacency list representation is utilized to maintain a memory-efficient network structure, ensuring rapid data retrieval and scalability for large-scale transit systems. The system also integrates a min-heap priority queue from the C++ Standard Template Library (STL) to optimize execution efficiency, drastically reducing the search space during pathfinding iterations in dynamic routing environments.

To enhance usability and practical application, a real-world simulation module is implemented, allowing users to input origin-destination matrices and visualize the optimal routes effectively. The project is structured as a modular programming pipeline, including network graph initialization, edge weight distribution, path computation, and path sequence generation components.

Experimental evaluations demonstrate that the proposed approach effectively handles simultaneous routing queries and successfully distinguishes between optimal and sub-optimal transit tracks under varying network complexities. This system contributes to improved urban mobility, reduced commuter travel times, and optimized route planning by enabling high-performance, real-time navigation calculations in complex transit networks.

TABLE OF CONTENTS

CHAPTER / SECTION	TITLE	PAGE NO.
Front Matter	List of Figures, Tables and Project Profile	6
Chapter 1	Introduction	7
1.1-1.6	Background, Problem Statement, Project Profile, Aim and Objectives	7
1.7-1.9	Outcomes, Scope and Significance	9
1.10-1.14	DSA Concepts, Requirements, Assumptions and Report Organization	10
Chapter 2	Literature Review and Theoretical Foundation	12
2.1-2.4	Route Planning, Graph Theory, Weighted Networks and Graph Storage	12
2.5-2.11	Priority Queues, Shortest Paths, Dijkstra's Algorithm and A*	14
2.12-2.17	Multi-Criteria Routing, Alternatives, Practices and Research Gap	16
Chapter 3	System Analysis, Design and Methodology	18
3.1-3.3	Development Method, Requirements, Dataset and Graph Construction	18
3.4-3.8	Architecture, Data Flow, Modular Design and Weight Models	19
3.9-3.14	Dijkstra, A*, Heuristics, Reconstruction and Route Evaluation	21
3.15-3.22	Cost Model, Validation, Complexity, Implementation and Verification	25
Chapter 4	Implementation, Testing, Results and Discussion	28
4.1-4.4	Implementation, Test Dataset, Functional Plan and Route Comparison	28
4.5-4.8	Correctness, Edge Cases, Performance Method and Results	30
4.9-4.14	Scalability, Route Quality, Validation, Outcomes and Limitations	32
Chapter 5	Conclusion, Limitations and Future Scope	34
Chapter 6	References	37

LIST OF FIGURES AND TABLES

ITEM	DESCRIPTION
Figure 2.1	Illustrative weighted road network
Figure 3.1	Layered architecture of the Smart Navigation System
Figure 3.2	End-to-end data flow for a navigation query
Figure 3.3	Core class relationship diagram
Figure 3.4	Dijkstra's algorithm flow
Figure 3.5	A* search flow
Figure 3.6	Multi-criteria route scoring model
Figure 4.1	Illustrative runtime trend
Figure 4.2	Trade-off between candidate routes
Table 1.1	Project profile
Table 2.1	Comparison of graph storage methods
Table 2.2	Comparison of shortest-path algorithms
Table 3.1	Core data entities
Table 3.2	Routing objective modes
Table 4.1	Functional test cases
Table 4.2	Illustrative route comparison
Table 4.3	Synthetic performance observations
Table 5.1	Proposed development roadmap

PROJECT PROFILE

FIELD	DETAIL
Project Topic Name	Smart Navigation System Using Graph Data Structure
Technology Used	C++, Graphs, Dijkstra's Algorithm, A* Algorithm and STL containers including priority_queue, vector, queue and map
Source of Project	Self-developed Academic Project
Primary Purpose	Find and compare the fastest, shortest and most cost-effective paths between two locations in a weighted road network.
Core Inputs	Source, destination, route preference, fuel price, vehicle mileage and optional heuristic coordinates.
Core Outputs	Recommended path, complete location sequence, distance, estimated travel time, estimated fuel/toll cost, efficiency score and algorithm statistics.

Table 1.1: Consolidated project profile based on the approved project form

CHAPTER 1
INTRODUCTION

1.1 Background and Context

Navigation is a practical application of graph theory in which a physical transportation network is converted into a mathematical structure that can be searched efficiently. Cities, road junctions, landmarks or service points are represented as vertices, while roads connecting them are represented as edges. Each edge may carry several measurable properties such as geographic distance, expected travel time, toll amount, fuel consumption, road quality or traffic intensity. Once this abstraction is created, a route-planning problem becomes a shortest-path or least-cost-path problem over a weighted graph.

A modern traveller rarely asks only for the route with the fewest kilometres. A short route may contain narrow roads, congestion or costly tolls; a longer route may be faster because it uses a highway; and a route that is fast for a car may be inefficient when fuel cost is the main concern. A useful academic navigation system should therefore separate the physical road network from the user's objective. The same graph can be evaluated with distance as the weight, time as the weight, monetary cost as the weight, or a weighted combination of several criteria.

The Smart Navigation System developed in this project demonstrates how core Data Structures and Algorithms concepts can be assembled into a complete software pipeline. The project uses an adjacency list to store a sparse road network, a minimum-priority queue to select promising locations, Dijkstra's algorithm to guarantee an optimal route for non-negative edge weights, and A* search to guide exploration toward the destination using a geographic heuristic. C++ Standard Template Library containers provide efficient, reliable building blocks for the implementation.

1.2 Problem Statement

The central problem is to design a program that accepts a valid source and destination, represents available roads accurately, computes an optimal route according to a selected objective, and explains the result in terms that are useful to a user. The program must support networks containing cycles, multiple alternatives, disconnected areas and roads with different metric values. It must avoid infinite traversal, reject invalid input, and reconstruct the complete path rather than reporting only a numerical cost.

Formally, consider a graph G = (V, E), where V is the set of locations and E is the set of directed or undirected road segments. For each edge e, the system stores distance d(e), estimated time t(e), monetary cost c(e), and any additional penalty required by the chosen model. For a path P consisting of a sequence of edges, the system must minimize an additive objective W(P) while preserving connectivity from a source s to a destination g. Depending on user preference, W(P) may represent total distance, total time, total cost or a composite score.

W(P) = sum of edge weights along P, for every edge belonging to the selected route

The design challenge extends beyond the mathematical search. Real software must map readable location names to internal identifiers, handle duplicate or missing data, maintain parent information for path reconstruction, calculate secondary metrics after the route is found, and present route trade-offs clearly. The project therefore treats navigation as a system-design problem supported by shortest-path algorithms, not merely as a single function that returns a number.

1.3 Need for the Proposed System

Manual route selection becomes unreliable when a network contains many intersections and several possible paths. Human judgement can compare two or three obvious routes, but it does not consistently evaluate hundreds or thousands of combinations. A greedy local choice such as selecting the nearest next junction can lead to a poor global route because the selected road may later connect to an expensive or slow segment. A graph-search algorithm evaluates cumulative cost and therefore reasons about the complete route rather than a single immediate step.

The project is also needed as an educational bridge between classroom theory and realistic application design. Graphs, heaps and maps are often learned independently. The Smart Navigation System shows how these structures cooperate: the adjacency list stores the topology, the map resolves names, the priority queue drives efficient exploration, and the parent array converts an optimal cost into a readable itinerary. Implementing two algorithms on the same graph further develops the ability to compare correctness, performance and suitability instead of memorizing one solution.

1.4 Project Topic, Technology and Source

The approved project topic is Smart Navigation System Using Graph Data Structure. It is a self-developed academic project implemented primarily in C++. The principal technologies are weighted graphs, adjacency lists, priority queues, Dijkstra's algorithm, A* search, and STL containers such as vector, priority_queue, queue and map or unordered_map. The road-network dataset used for demonstration is manually constructed or synthetically generated so that algorithm behaviour can be tested in a controlled environment without depending on an external commercial mapping service.

1.5 Aim of the Project

The aim is to design, implement and evaluate a modular Smart Navigation System that finds an optimal path between two locations and compares route alternatives using distance, travel time, fuel or toll cost, and overall efficiency. The system should remain understandable enough for academic evaluation while using data structures and algorithmic techniques that are directly relevant to larger navigation systems.

1.6 Objectives of the Study

- Represent cities, junctions and landmarks as vertices in a weighted graph and represent roads as edges containing multiple navigation metrics.
- Store the network through an adjacency list so that memory usage grows with actual roads rather than every possible pair of locations.
- Implement Dijkstra's algorithm with a min-heap priority queue to obtain an exact minimum-cost route for non-negative edge weights.
- Implement A* search with an admissible geographic heuristic to reduce unnecessary exploration in point-to-point queries.
- Allow route selection by shortest distance, fastest estimated time, minimum monetary cost and balanced multi-criteria efficiency.
- Calculate complete route metrics, reconstruct the ordered path and present alternatives in a transparent comparison format.
- Validate source, destination, graph connectivity and metric values, and handle unreachable or invalid queries without program failure.

- Measure time complexity, memory usage, explored nodes and observed query latency on controlled sparse graphs.
- Develop a modular C++ design that can later be connected to real map data, traffic updates or a graphical interface.

1.7 Expected and Achieved Project Outcomes

The project outcomes are defined in terms of implementation capability and learning value. First, the system implements graph-based route optimization through both Dijkstra's and A* algorithms. Second, it compares multiple routes using distance, travel time, fuel cost and an efficiency score instead of presenting a route without explanation. Third, it strengthens practical understanding of graph representation, priority queues, shortest-path relaxation, heuristics and STL usage. Finally, it demonstrates how a real navigation requirement can be decomposed into data modelling, algorithm design, validation, testing and user-oriented output.

1. A working graph-search module that executes Dijkstra's algorithm and A* search on the same network.

2. A route-comparison module that reports the effect of selecting distance, time, cost or balanced preference.

3. A reusable graph class supporting locations, roads, directed or undirected insertion and name-to-ID lookup.

4. A structured testing report covering normal routes, cycles, equal-cost alternatives, invalid input and unreachable destinations.

5. A documented foundation for future features such as traffic-aware routing, real geographic data and mobile presentation.

1.8 Scope of the Project

The implemented scope includes an in-memory weighted road graph, source and destination selection, four routing objectives, exact Dijkstra routing, heuristic-guided A* routing, path reconstruction, secondary metric calculation and text-based result presentation. The graph may be loaded from hard-coded data or a structured text file. Each location can store coordinates so that A* can estimate remaining distance. Each road can store distance, travel time and cost independently.

The project focuses on a static snapshot of the road network during a query. It does not claim to reproduce a production navigation platform. Live traffic, turn restrictions, lane-level guidance, GPS tracking, map rendering and continuously changing travel times are outside the current implementation. These limitations are deliberate: they allow the report to examine fundamental DSA concepts deeply and to evaluate algorithm behaviour without hiding it behind a large external framework.

1.9 Significance of the Study

The significance of the work lies in combining algorithmic correctness with practical interpretability. Dijkstra's algorithm provides a dependable baseline, while A* introduces the important idea that domain knowledge can guide a search without sacrificing optimality when the heuristic is well designed. The multi-criteria model demonstrates that the definition of optimality is a design decision. A route is optimal only with respect to a specified objective and reliable input weights.

For students, the project provides evidence of applied problem solving. It requires careful use of references, const-correctness, STL comparators, numeric limits, parent arrays, input normalization

and test design. For a future developer, the same architecture can be extended with real datasets or service APIs. For evaluators, the system exposes intermediate concepts such as explored nodes and metric totals, allowing the result to be verified rather than accepted as a black box.

1.10 DSA Concepts Used

CONCEPT	ROLE IN THE PROJECT
Graphs	Model the complete navigation network and its connectivity.
Weighted Graphs	Associate every road with distance, time, cost or a composite weight.
Adjacency List	Store only existing outgoing roads for each location.
Priority Queue / Min Heap	Select the unsettled location with the smallest known score.
Dijkstra's Algorithm	Guarantee an exact shortest path under non-negative weights.
A* Search Algorithm	Use g(n) + h(n) to guide the search toward the destination.
Vector	Store adjacency lists, distances, parents and route sequences.
Queue	Support optional breadth-first validation and traversal utilities.
Map / unordered_map	Translate human-readable location names into numeric IDs.
Greedy Relaxation	Improve tentative path costs whenever a cheaper predecessor is found.

1.11 Functional Requirements

- Create, update and inspect a road graph containing unique locations and validated road metrics.
- Accept source, destination and optimization preference from the user.
- Select Dijkstra or A* and execute the algorithm without modifying the original graph.
- Return the ordered path, aggregate distance, aggregate time, cost estimate and efficiency score.
- Display a clear message when no path exists or when a location name is invalid.
- Support repeated queries without rebuilding the graph for every request.
- Record diagnostic values such as number of expanded nodes, relaxations and query time for analysis.

1.12 Non-Functional Requirements

Correctness is the highest-priority requirement: the selected route must minimize the active objective under the stated assumptions. Efficiency requires adjacency-list storage and heap-based selection so that sparse networks remain practical. Maintainability requires separation between graph storage, routing algorithms, metric evaluation and user interaction. Robustness requires numeric overflow protection, positive metric checks and safe handling of missing paths. Usability requires readable names and an explanation of why a route was recommended. Portability requires standard C++ and STL features rather than platform-specific dependencies.

1.13 Assumptions and Constraints

- All routing weights are non-negative. Negative distance, travel time or monetary cost is rejected as invalid data.
- Coordinates used by A* are meaningful and are converted to a lower-bound estimate compatible with the selected objective.

- A road is treated as undirected only when travel in both directions is allowed; otherwise separate directed edges are inserted.
- Estimated time and cost are simplified values for academic testing and may differ from real traffic or vehicle behaviour.
- The complete graph fits in main memory and is not modified concurrently while an algorithm is processing a query.
- Benchmark values are illustrative results from controlled synthetic graphs and should not be interpreted as commercial performance claims.

1.14 Organization of the Report

Chapter 1 defines the problem, objectives, scope, requirements and expected outcomes. Chapter 2 reviews graph theory, shortest-path methods, Dijkstra's algorithm, A* search, heuristics and multi-criteria routing. Chapter 3 presents the development methodology, data structures, architecture, algorithms, equations, pseudocode and C++ implementation design. Chapter 4 explains testing, route comparisons, performance observations, correctness validation and limitations. Chapter 5 concludes the work and proposes a staged future roadmap. Chapter 6 lists the books, research papers and technical references supporting the study.

CHAPTER 2
LITERATURE REVIEW AND THEORETICAL FOUNDATION

2.1 Introduction to Route-Planning Research

Route planning is one of the best-known applications of graph algorithms because it converts a familiar real-world task into a precise optimization problem. The basic task is point-to-point shortest-path search, but practical navigation introduces several variants: single-source search, all-pairs preprocessing, time-dependent travel, multiple objectives, turn restrictions and dynamic network updates. The most appropriate algorithm depends on graph size, weight properties, query frequency, availability of geographic coordinates and whether an exact optimum is mandatory.

Classical shortest-path research established general algorithms that remain relevant in modern systems. Dijkstra's method is the standard exact algorithm for non-negative weights. Bellman-Ford supports negative edges at a higher computational cost. Floyd-Warshall computes all-pairs distances for comparatively small dense graphs. A* adds a heuristic estimate and is especially suitable when one destination is known. Large commercial networks often apply preprocessing methods such as contraction hierarchies, landmark-based A* or multilevel partitioning, but these advanced methods build on the same graph and relaxation principles studied in this project.

2.2 Graph Theory for Navigation Networks

A graph G = (V, E) contains a vertex set V and an edge set E. In this project, a vertex represents a city, junction or named location. An edge represents a road segment that can be traversed from one endpoint to another. A road network can be undirected when both travel directions are equivalent, directed when one-way movement is possible, or mixed when some roads are bidirectional and others are directional. The distinction is important because an undirected road is stored as two directed adjacency entries in a typical implementation.

A path is a sequence of vertices in which each consecutive pair is connected by an edge. A simple path does not repeat vertices, although the graph itself may contain cycles. A cycle is not inherently an error; real roads naturally form loops. The algorithm must therefore keep best-known scores and avoid endlessly revisiting expensive states. Connectivity describes whether a path exists between locations. A disconnected component represents an isolated region, data omission or temporary road closure and must result in a graceful unreachable response.

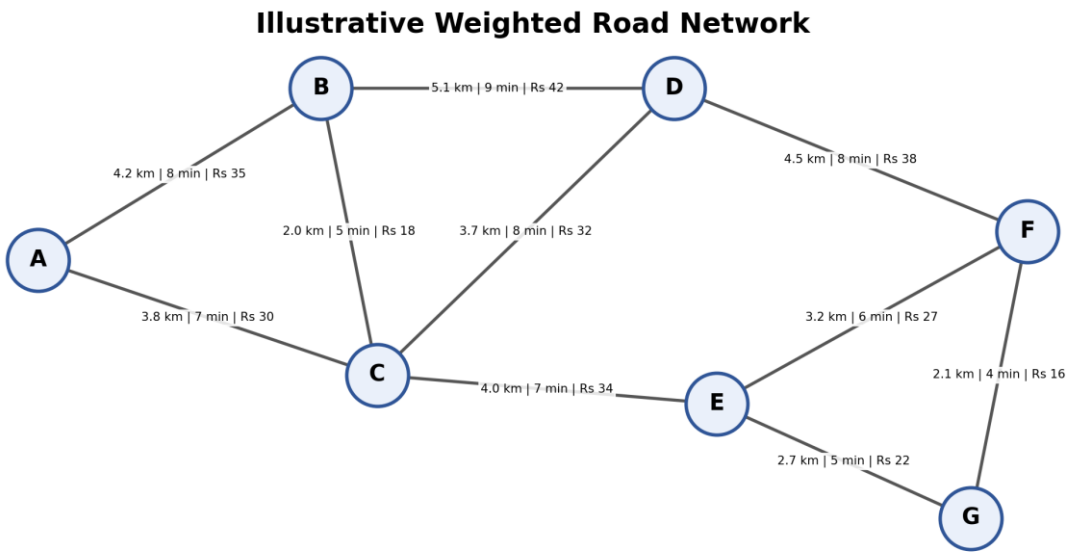


Figure 2.1: Illustrative weighted road network with distance, time and cost attributes

2.3 Weighted Graph Model

An unweighted graph treats every road as having equal cost and therefore minimizes the number of edges. This is rarely sufficient for navigation. Weighted graphs assign a numerical value to each edge, allowing the algorithm to minimize a meaningful quantity. Distance weights may be measured in kilometres, time weights in minutes, and monetary weights in rupees. The units must remain consistent inside a single algorithm run. Adding kilometres directly to rupees without normalization would produce an arbitrary score that has no clear interpretation.

The same road can therefore expose multiple attributes while the routing engine receives one scalar weight through a policy function. This design prevents duplication of graph data. A distance query returns d(e); a time query returns t(e); a cost query returns c(e); and a balanced query computes a normalized weighted score. Secondary metrics are still summed along the selected path so that the user can see trade-offs. The separation between stored attributes and active weight is central to the project architecture.

2.4 Adjacency Matrix and Adjacency List

Graph representation affects both memory consumption and traversal cost. An adjacency matrix allocates a V by V table. It provides constant-time edge existence checks but consumes O(V^2) memory even when very few location pairs are directly connected. Road networks are normally sparse because a junction connects only to nearby roads rather than every other junction. An adjacency list stores a collection of outgoing edges for each vertex and therefore requires O(V + E) storage.

CRITERION	ADJACENCY MATRIX	ADJACENCY LIST
Memory	O(V^2)	O(V + E)
Enumerating neighbours	O(V)	O(degree(u))
Testing one edge	O(1)	O(degree(u)) unless indexed
Suitability	Small dense graphs	Large sparse road networks
Implementation in project	Not selected	Selected using vector<vector<Edge>>

Table 2.1: Comparison of graph storage methods

The adjacency list also supports a natural C++ design. The outer vector index is a location identifier. Each inner vector stores Edge objects containing destination and metrics. Iterating over graph.adj[u] visits only actual roads leaving u, which is exactly the operation performed during relaxation. The structure can be converted to compressed arrays or database pages in a larger system without changing the mathematical interface.

2.5 Priority Queues and the Min-Heap

Both Dijkstra and A* repeatedly require the unsettled node with the smallest score. A linear scan over all vertices can perform this selection in O(V) time, leading to O(V^2) behaviour. A minimum-priority queue reduces selection and insertion to logarithmic time. The C++ priority_queue is a max-heap by default, so the implementation supplies greater<pair<...>> or a custom comparator to obtain minimum-first behaviour.

A common and simple implementation does not decrease an existing heap key in place. Instead, it pushes a new pair whenever a better score is found. Older entries remain in the heap and become stale. When an entry is extracted, the algorithm compares its stored score with the current best score and discards it if they differ. This lazy-update technique is correct, reduces implementation complexity, and has the standard O((V + E) log V) sparse-graph bound up to constant factors.

2.6 The Shortest-Path Problem and Relaxation

For a source s, the tentative value dist[v] is an upper bound on the best route from s to v discovered so far. Initially dist[s] is zero and all other values are infinity. When edge (u, v) with weight w is examined, the algorithm tests whether reaching v through u is cheaper than the current value. This operation is called relaxation. If the candidate is better, dist[v] and parent[v] are updated.

$$candidate = dist[u] + w(u,v); \text{ update when } candidate < dist[v]$$

Relaxation is local, but repeated relaxations propagate globally optimal information through the graph. The parent array records the predecessor that produced each improvement. After the destination is settled, the system follows parents from destination to source and reverses the sequence. Without this additional structure, the algorithm would know the minimum score but could not explain the actual route.

2.7 Dijkstra's Algorithm

Dijkstra's algorithm is a greedy single-source shortest-path method for graphs with non-negative weights. At each step, it selects the unsettled vertex with the minimum tentative distance and permanently settles that value. The key reasoning is that no later path can reach the selected vertex more cheaply: any alternative would first have to pass through another unsettled vertex whose tentative cost is at least as large, and all remaining edges add a non-negative amount.

For a point-to-point query, the algorithm may stop when the destination is extracted with its current best score. It is unnecessary to settle the remaining graph because the destination value is already final. However, if distances from the source to every vertex are needed, the search continues until the heap is empty. This difference illustrates how query requirements affect practical execution even when theoretical worst-case complexity is unchanged.

2.8 Correctness Conditions for Dijkstra

The non-negative-weight assumption is not a minor implementation detail; it is the foundation of the greedy proof. If a negative road weight were allowed, an apparently final vertex might later be

improved through a route containing the negative edge. Physical distance and travel time are naturally non-negative, while monetary rewards or credits should not be represented as negative routing edges without changing the algorithm. Dataset validation therefore enforces non-negative active weights before a query begins.

Correctness also depends on numeric safety and graph consistency. Infinity must be represented with enough headroom that adding a valid weight does not overflow. Parent values must be initialized to an invalid marker. Location identifiers used by edges must exist. In undirected graphs, both directions should be inserted consistently. These engineering conditions connect the mathematical proof to a reliable implementation.

2.9 A* Search Algorithm

A* is a best-first shortest-path algorithm that evaluates a node n using f(n) = g(n) + h(n). The value g(n) is the exact cost from the source to n discovered so far. The heuristic h(n) estimates the remaining cost from n to the destination. Dijkstra is recovered as the special case h(n) = 0 for every node. A useful heuristic gives the search directional knowledge, allowing it to avoid exploring large regions that are unlikely to lead efficiently toward the goal.

$$f(n) = g(n) + h(n)$$

A* is especially appropriate for point-to-point navigation because the destination is known and location coordinates are often available. For shortest-distance routing, straight-line or great-circle distance is a natural lower bound when road distance cannot be less than direct geometric distance. For fastest-time routing, direct distance divided by an upper-bound travel speed can provide a time lower bound. The heuristic must be expressed in compatible units and must not overestimate the true remaining optimal cost if an exact route is required.

2.10 Admissible and Consistent Heuristics

A heuristic is admissible when h(n) never exceeds the true minimum remaining cost from n to the goal. Admissibility ensures that A* does not discard the optimal route by making it appear too expensive. A heuristic is consistent when h(u) <= w(u,v) + h(v) for every edge. Consistency implies that f-values do not decrease along a path and supports efficient closed-set behaviour. Straight-line distance is consistent for ordinary distance graphs when edge distance is at least geometric separation.

A heuristic that is too weak approaches zero and provides little improvement over Dijkstra. A heuristic that is aggressive but overestimates may reduce search work while returning a suboptimal route. The project therefore favours a conservative lower-bound heuristic. It records explored-node counts so that the practical effect can be compared without compromising correctness.

2.11 Dijkstra and A*: Comparative Analysis

ASPECT	DIJKSTRA	A*
Guidance	No destination heuristic	Uses estimate toward destination
Optimality	Exact for non-negative weights	Exact with admissible heuristic
Best use	Baseline, all destinations, missing coordinates	Point-to-point geographic search
Worst-case complexity	O((V+E) log V)	O((V+E) log V) in worst case
Typical exploration	May expand nodes in all directions	Often expands a corridor toward goal
Data requirement	Weights only	Weights plus heuristic data

ASPECT	DIJKSTRA	A*
Failure risk	Negative weight invalidates proof	Overestimating/incompatible heuristic may harm optimality

Table 2.2: Comparison of the principal routing algorithms

The comparison does not imply that A* is universally superior. Dijkstra is easier to validate, works without coordinates and provides all-source distances if allowed to finish. A* is usually more targeted for a single destination, but its advantage depends on heuristic quality and network structure. Implementing both methods provides a trustworthy baseline and makes performance observations more meaningful.

2.12 Multi-Criteria Route Optimization

Real navigation criteria have different units and priorities. A multi-criteria system can return several Pareto-optimal alternatives, where no route is better in every metric, or it can convert preferences into one scalar objective. The project uses scalarization for algorithm simplicity while still displaying all raw totals. Metrics are normalized against reference values or meaningful scales, multiplied by user preference weights, and summed to create a composite edge score.

$$score(e) = wD * Dnorm(e) + wT * Tnorm(e) + wC * Cnorm(e) + wE * Enorm(e)$$

The preference weights are non-negative and sum to one. For a shortest route, wD is one and the remaining weights are zero. For a balanced route, the weights may distribute importance across distance, time and cost. Because scalarization can hide trade-offs, the interface reports the original totals and may compute several objective-specific alternatives before recommending one. This preserves transparency and prevents the efficiency score from being mistaken for a physical unit.

2.13 Fuel-Cost and Efficiency Models

A simplified fuel estimate can be calculated from path distance, vehicle mileage and fuel price. If distance is measured in kilometres and mileage in kilometres per litre, estimated fuel consumed is distance divided by mileage. Multiplying by price per litre gives fuel cost. Toll charges stored on road edges may then be added. This model does not include acceleration, elevation, idling or vehicle-specific engine curves, so the result is an academic estimate rather than a billing value.

$$fuel\ cost = (route\ distance / vehicle\ mileage) * fuel\ price + total\ toll$$

Efficiency can be presented as a relative score comparing normalized metrics among candidate routes. A lower composite cost indicates a better match for the selected preference. The system avoids labelling one route universally best; instead, it states that a route is recommended under a particular objective and set of assumptions.

2.14 Alternative Algorithms

Breadth-first search finds the path with the fewest edges in an unweighted graph but cannot correctly minimize unequal road costs. Bellman-Ford supports negative weights and detects negative cycles but requires O(VE) time, making it unnecessary for non-negative road data. Floyd-Warshall uses O(V^3) time and O(V^2) memory to compute all-pairs paths and is useful for small dense graphs or repeated queries after preprocessing. Bidirectional Dijkstra searches from both endpoints and can reduce the explored region when reverse edges are available.

Greedy best-first search uses only h(n), which can be fast but does not account for cost already incurred and therefore lacks A*'s optimality guarantee. Advanced road-routing systems use

contraction hierarchies, reach, landmarks, transit-node routing or customizable route planning. These methods are outside the implementation scope but are relevant future directions once the base graph model and correctness tests are stable.

2.15 Review of Implementation Practices

Literature on algorithm engineering emphasizes that asymptotic complexity is only one part of performance. Data locality, heap behaviour, edge representation, memory allocation and early termination affect observed query time. A vector of contiguous Edge objects is usually preferable to a linked list for traversal because it improves cache locality and reduces allocation overhead. Passing the graph by const reference avoids copying. Reserving vector capacity during loading can further reduce reallocations.

Reliable pathfinding software also needs deterministic testing. Small hand-verifiable graphs reveal logical mistakes, while randomized graphs allow broad comparison against a trusted baseline. The selected path cost should be cross-checked with an independent implementation or exhaustive enumeration on very small graphs. Performance tests should separate graph construction from query execution and should report environment and dataset assumptions.

2.16 Research Gap Addressed by the Project

Many introductory projects demonstrate only Dijkstra's distance array on a small graph. They do not compare algorithms, reconstruct a user-readable route, model several road metrics, validate heuristic conditions or explain why a route is suitable. At the opposite extreme, production mapping platforms are too complex and closed for a student to inspect end to end. This project addresses the middle ground by building a transparent, modular prototype that remains academically manageable while representing the important decisions found in a navigation pipeline.

The contribution is therefore educational and architectural rather than a claim of a new shortest-path algorithm. The work shows how known algorithms can be applied correctly, compared fairly, and extended with route preferences. The detailed design and testing approach make the project reproducible and provide a foundation for future integration with geographic datasets.

2.17 Chapter Summary

This chapter established the theoretical basis for the Smart Navigation System. A road network is naturally represented as a sparse weighted graph stored through adjacency lists. A min-heap supports efficient node selection. Dijkstra provides an exact baseline for non-negative weights, while A* uses an admissible heuristic to focus point-to-point exploration. Multi-criteria routing requires careful normalization and transparent reporting. These principles guide the system design presented in Chapter 3.

CHAPTER 3
SYSTEM ANALYSIS, DESIGN AND METHODOLOGY

3.1 Development Methodology

The project follows an iterative engineering methodology. The first iteration defines a small graph and verifies basic shortest-distance routing. The second introduces reusable Location, Edge and Graph structures. The third adds path reconstruction and input validation. The fourth introduces A* and coordinate-based heuristics. The fifth adds time, cost and balanced objectives. The final iteration performs systematic tests, records algorithm statistics and documents limitations. This incremental process reduces debugging difficulty because each new capability is tested against a stable baseline.

The methodology separates requirement analysis, data modelling, algorithm design, implementation and evaluation. Each phase produces a concrete artefact: requirements and assumptions; graph schema; algorithm pseudocode; C++ modules; and test evidence. Although the application is not a machine-learning system, the term dataset is used for the collection of locations and roads. There is no training phase; instead, graph construction and validation prepare the data for deterministic search.

3.2 Requirement Analysis

A navigation query is defined by source, destination, objective and optional vehicle parameters. The system must return a connected path minimizing the active objective. It should also calculate distance, time and cost along that path even when those values were not minimized. The requirements imply that edge data must retain all metrics, the routing algorithm must accept a weight policy, and the result object must contain both the route sequence and diagnostic information.

The analysis also identifies failure conditions. A source or destination may be unknown; both may refer to the same location; no connection may exist; an edge may contain an invalid negative value; coordinates may be missing; or the composite weights may not be normalized. The design responds to these cases through validation and fallback rules. For example, A* falls back to a zero heuristic when coordinates are unavailable, producing Dijkstra-equivalent behaviour rather than an incorrect estimate.

3.3 Dataset and Graph Construction

The academic dataset consists of location records and road records. A location record contains an integer identifier, unique name and optional x-y or latitude-longitude coordinates. A road record contains from-ID, to-ID, distance, estimated travel time, toll or fixed cost, and a flag indicating whether the road is bidirectional. Data may be embedded in code for a demonstration or loaded from comma-separated values for easier modification.

ENTITY	IMPORTANT FIELDS	PURPOSE
Location	id, name, x, y	Identify a graph vertex and support lookup and heuristic calculation.
Edge	to, distance, time, toll, roadType	Represent one directed road and all measurable attributes.
Preferences	distanceWeight, timeWeight, costWeight	Define the active objective or balanced profile.
RouteResult	path, distance, time, cost, score, expandedNodes	Return both user-facing and diagnostic output.

ENTITY	IMPORTANT FIELDS	PURPOSE
Graph	adjacency list, location array, name map	Own the network and expose validated neighbour access.

Table 3.1: Core data entities

During preprocessing, names are trimmed and converted to a normalized form for lookup. Duplicate names or IDs are rejected unless the design explicitly supports disambiguation. For every valid undirected road, two directed Edge entries are inserted. A connectivity scan may be executed after loading to identify isolated regions. Edge metrics are checked for finite non-negative values. Coordinates are checked for valid range when latitude and longitude are used.

3.4 System Architecture

The system uses a layered architecture so that user interaction, graph storage, search logic and metric evaluation can evolve independently. The presentation layer reads input and prints results. The validation layer converts names to IDs and checks preferences. The graph layer owns locations and road edges. The routing layer runs Dijkstra or A*. The evaluation layer aggregates all metrics and ranks alternatives. The architecture avoids placing console input inside the algorithm, making the search functions reusable in a future web or mobile interface.

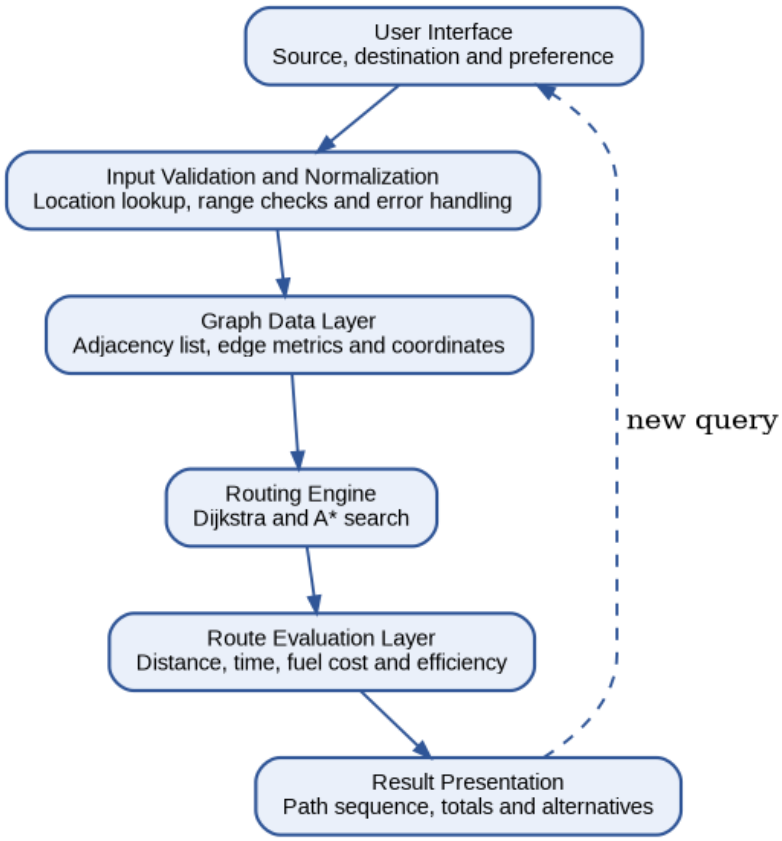


Figure 3.1: Layered architecture of the Smart Navigation System

3.5 End-to-End Query Flow

A query begins when the user selects a source, destination and optimization mode. The names are normalized and mapped to vertex IDs. Distance, parent and state arrays are initialized. The selected algorithm then explores the graph and either settles the destination or exhausts all reachable nodes.

The route is reconstructed, and the evaluator traverses each consecutive edge to sum distance, time and cost. When alternative objectives are requested, the pipeline repeats the search with different weight policies and compares the resulting RouteResult objects.

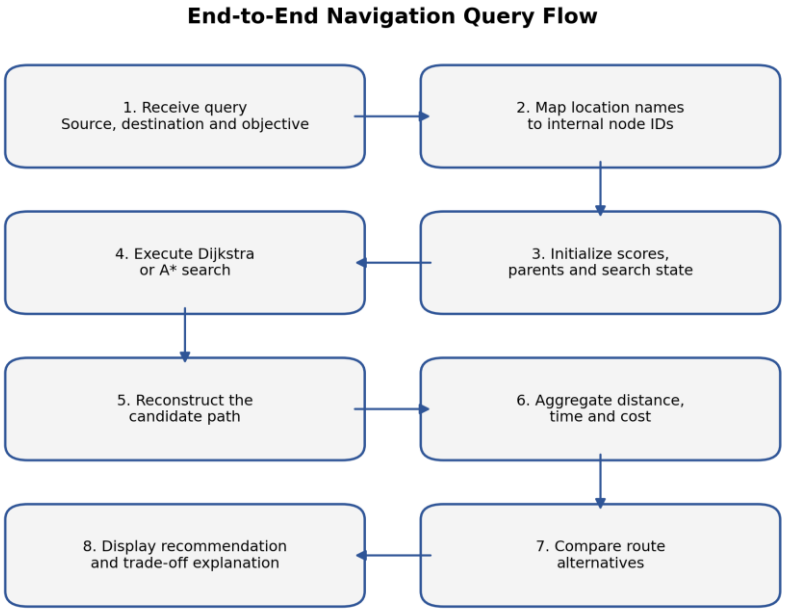


Figure 3.2: End-to-end data flow for a navigation query

3.6 Object-Oriented and Modular Design

The Graph class is responsible only for network integrity and neighbour access. Router holds a const reference to Graph and implements search functions without owning or modifying the network. RouteEvaluator computes totals and scores from a returned path. RouteResult is a value object that carries output. This division follows the single-responsibility principle and makes unit testing easier. A graph test can verify road insertion independently from an algorithm test, while an evaluator test can use a manually specified path.

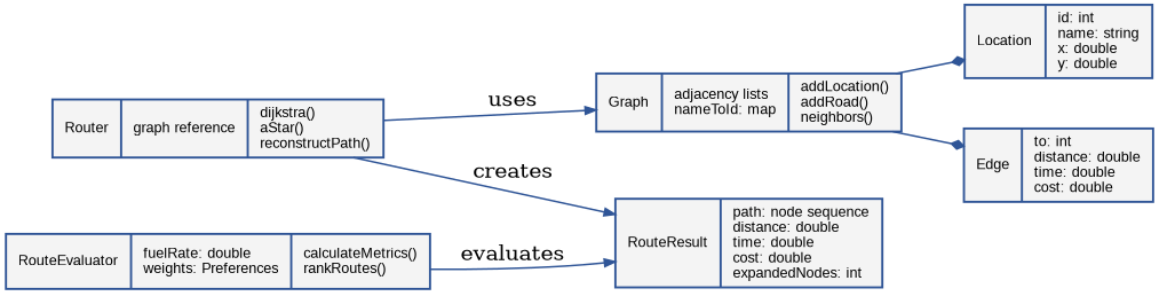


Figure 3.3: Core class relationship diagram for the C++ implementation

3.7 Edge-Weight Selection Strategy

The routing engine obtains a scalar weight for every edge through an objective function. For shortest routing, the function returns edge.distance. For fastest routing, it returns edge.time. For minimum-cost routing, it returns estimated fuel cost for the segment plus toll. For balanced routing, it returns a normalized composite score. A function object or lambda can express this policy, allowing Dijkstra and A* to share one traversal structure.

MODE	ACTIVE EDGE WEIGHT	INTERPRETATION
Shortest	distance	Minimum kilometres; may not be fastest.
Fastest	estimated time	Minimum expected minutes under static estimates.
Lowest cost	fuel estimate + toll	Minimum simplified monetary expense.
Balanced	weighted normalized score	Trade-off controlled by preference weights.
Diagnostic	unit edge weight	Minimum number of road segments for comparison only.

Table 3.2: Routing objective modes

3.8 Normalization for Balanced Routing

Distance, time and rupees cannot be combined safely without scaling. The project uses reference factors Dref, Tref and Cref chosen from dataset statistics or user-understandable units. Each edge metric is divided by its reference factor to produce dimensionless values. Preference weights are then applied. The reference factors affect sensitivity and must be documented. An alternative is min-max normalization over all edges, but that approach changes when outlier roads are added. Fixed reference factors provide more stable interpretation for an academic dataset.

$$balanced\ weight(e) = wD*d(e)/Dref + wT*t(e)/Tref + wC*c(e)/Cref$$

The weights wD, wT and wC are checked to be non-negative. They are divided by their sum if the user enters values that do not total one. When all values are zero, the request is rejected or replaced with an explicit default profile. The result page displays the preference profile so that the recommendation is reproducible.

3.9 Dijkstra's Algorithm Design

The Dijkstra implementation stores pairs of current score and vertex ID in a min-priority queue. The distance vector is initialized to infinity, parent values to -1, and the source score to zero. At extraction, a stale entry is skipped when its score is greater than the current distance. Each outgoing edge is converted to the active scalar weight, checked for validity and relaxed. The number of extracted nodes and successful relaxations is recorded for evaluation.

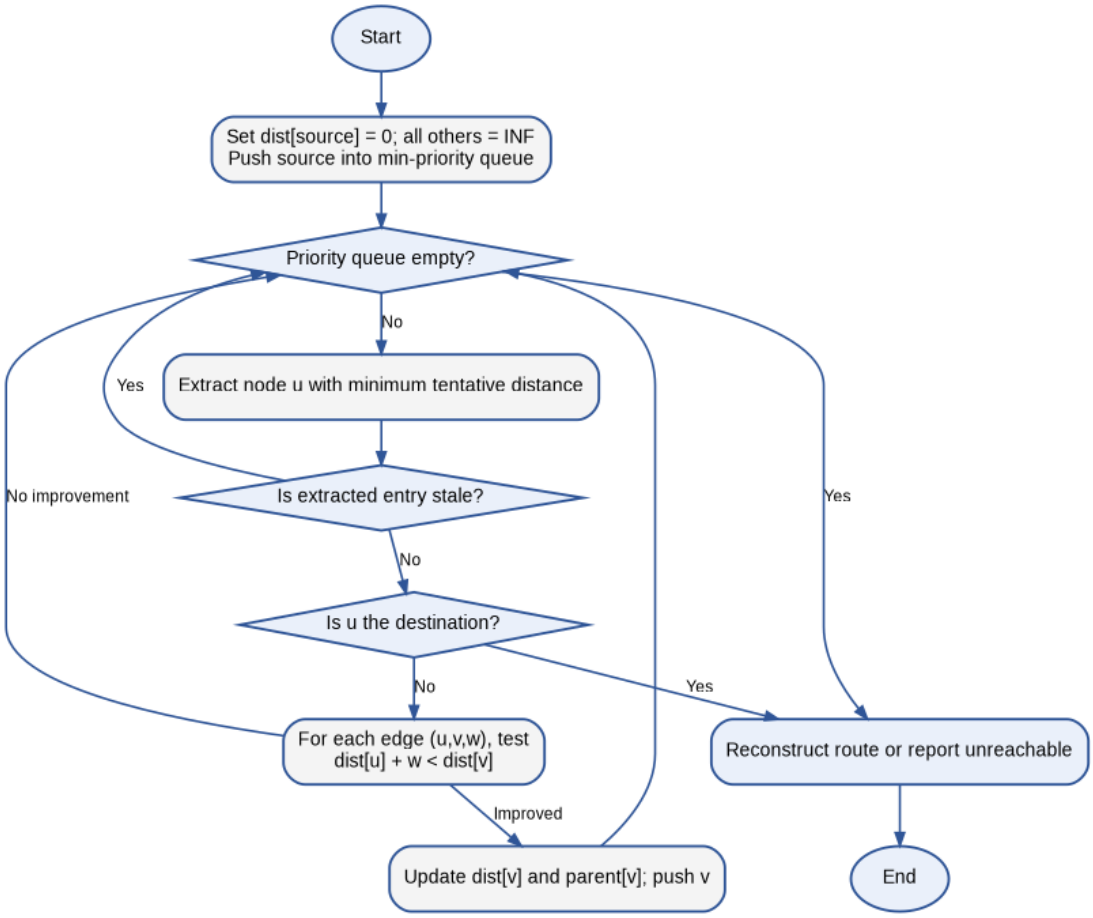


Figure 3.4: Flow of Dijkstra's shortest-path algorithm

```
RouteResult dijkstra(const Graph& graph, int source, int goal,
                     const WeightFunction& weightOf) {
    const double INF = numeric_limits<double>::infinity();
    vector<double> dist(graph.size(), INF);
    vector<int> parent(graph.size(), -1);
    priority_queue<State, vector<State>, greater<State>> open;

    dist[source] = 0.0;
    open.push({0.0, source});

    while (!open.empty()) {
        auto [score, u] = open.top();
        open.pop();
        if (score > dist[u]) continue;          // stale entry
        if (u == goal) break;                   // goal is settled

        for (const Edge& e : graph.neighbours(u)) {
            const double w = weightOf(e);
            const double candidate = dist[u] + w;
            if (candidate < dist[e.to]) {
                dist[e.to] = candidate;
                parent[e.to] = u;
                open.push({candidate, e.to});
            }
        }
    }
    return buildResult(graph, source, goal, dist, parent);
}
```

Listing 3.1: Simplified C++ structure of heap-based Dijkstra routing

3.10 Dijkstra Invariants and Early Termination

The implementation maintains three important invariants. First, dist[v] is always the cost of some discovered path or infinity. Second, parent[v] identifies the predecessor of the path represented by dist[v]. Third, when a vertex is extracted with a non-stale minimum score, its value is final under non-negative weights. Early termination is therefore valid only after the destination is extracted, not when it is first inserted into the heap. A route discovered early may later be improved before settlement.

The algorithm also guards against invalid source and destination IDs before allocating work. If source equals destination, it immediately returns a one-location path with zero distance, time and cost. If the heap becomes empty while the destination distance remains infinity, the system returns an unreachable status and an empty path rather than attempting parent reconstruction.

3.11 A* Search Design

A* uses the same adjacency list and relaxation logic but orders the priority queue by f = g + h. The gScore vector stores the exact discovered cost, while the queue key contains the estimated total cost. A parent update occurs only when tentativeG improves gScore[v]. The heuristic function receives the current location, goal location and active objective. For distance mode, it returns geometric straight-line distance. For time mode, it divides geometric distance by a declared maximum feasible speed. For a balanced objective, the conservative implementation may use only the distance component or return zero when a valid lower bound cannot be guaranteed.

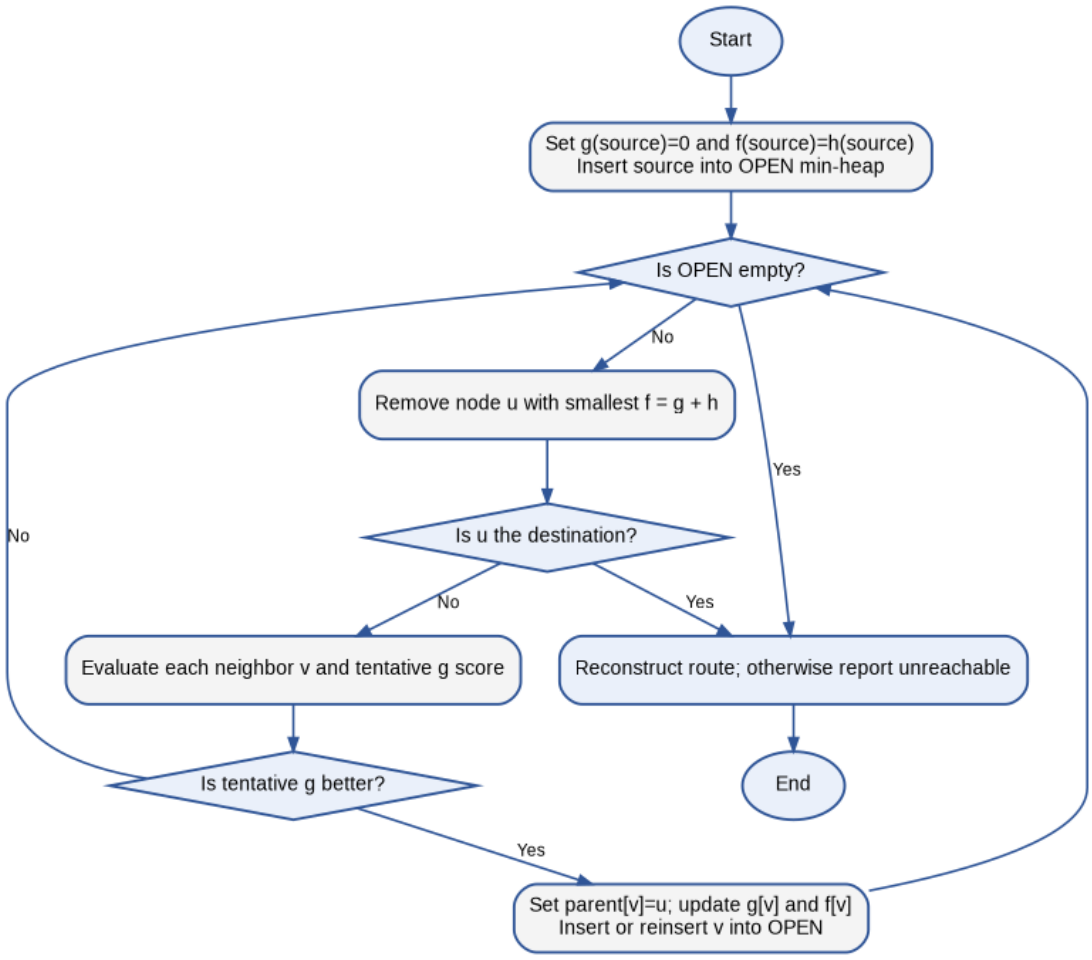


Figure 3.5: Flow of A point-to-point search*

```
RouteResult aStar(const Graph& graph, int source, int goal,
                  const WeightFunction& weightOf,
                  const HeuristicFunction& heuristic) {
    vector<double> g(graph.size(), INF);
    vector<int> parent(graph.size(), -1);
    priority_queue<State, vector<State>, greater<State>> open;

    g[source] = 0.0;
    open.push({heuristic(source, goal), source});

    while (!open.empty()) {
        auto [fScore, u] = open.top();
        open.pop();
        if (u == goal) break;

        for (const Edge& e : graph.neighbours(u)) {
            double tentative = g[u] + weightOf(e);
            if (tentative < g[e.to]) {
                g[e.to] = tentative;
                parent[e.to] = u;
                double f = tentative + heuristic(e.to, goal);
                open.push({f, e.to});
            }
        }
    }
    return buildResult(graph, source, goal, g, parent);
}
```

Listing 3.2: Simplified A structure using g-score and heuristic priority*

3.12 Heuristic Calculation

For planar demonstration coordinates, Euclidean distance is calculated from the difference in x and y values. For latitude-longitude data, the Haversine formula can estimate great-circle distance. The implementation should not mix degrees directly with kilometre road weights. Coordinates must be converted into a compatible distance estimate. The project demonstration uses a coordinate scale defined with the dataset so that heuristic distance is a conservative lower bound.

$$hDistance(n) = sqrt((xGoal - xN)^2 + (yGoal - yN)^2)$$

$$hTime(n) = hDistance(n) / maximum\ assumed\ speed$$

The maximum-speed denominator must be high enough that estimated remaining time does not exceed the true fastest road time. When this guarantee is uncertain, the safest exact choice is $h = 0$. The system exposes heuristic choice in configuration and validates A* costs against Dijkstra on test cases. Matching route cost provides strong evidence that the heuristic has not broken optimality, although formal admissibility still depends on data semantics.

3.13 Path Reconstruction

After a successful search, the parent array is followed from goal to source. Each vertex is appended to a reverse sequence. If a parent becomes -1 before the source is reached, the result is treated as invalid or unreachable. The sequence is reversed to produce source-to-destination order. For every consecutive pair, the evaluator retrieves the actual Edge used so that distance, time and cost can be accumulated. In graphs containing parallel roads between the same locations, the parent structure may also store the selected edge index rather than only the predecessor vertex.

```
vector<int> reconstructPath(int source, int goal,
                            const vector<int>& parent) {
    vector<int> path;
    for (int v = goal; v != -1; v = parent[v]) {
        path.push_back(v);
        if (v == source) break;
```

```
    }
    if (path.empty() || path.back() != source) return {};
    reverse(path.begin(), path.end());
    return path;
}
```

Listing 3.3: Parent-based reconstruction of the ordered route

3.14 Multi-Criteria Route Evaluation

The route evaluator performs a second pass over a candidate path. It calculates raw totals independently from the scalar search score. This is important because a balanced score is dimensionless and cannot replace understandable metrics. For every road, the evaluator adds physical distance, expected minutes, toll, fuel estimate and any road penalty. It then derives average speed, cost per kilometre and a relative efficiency rating when meaningful.

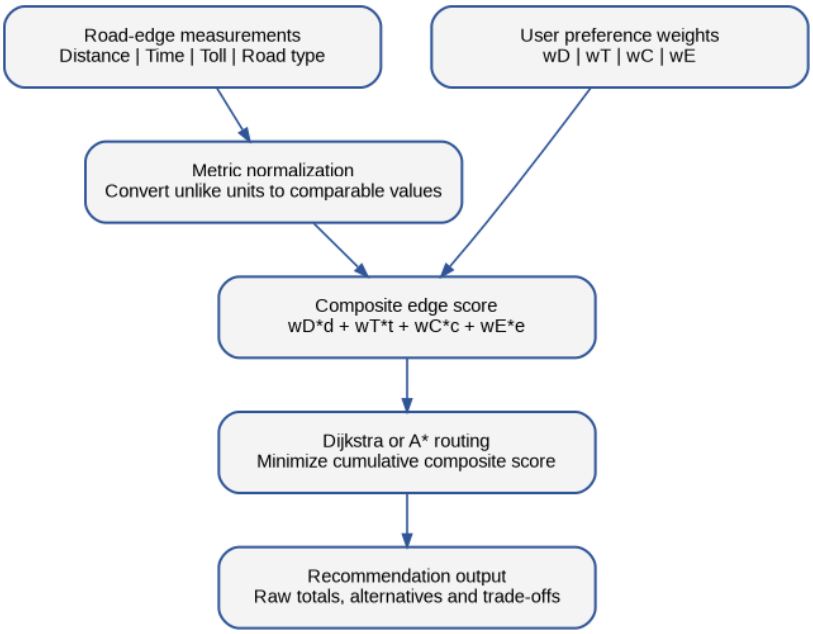


Figure 3.6: Multi-criteria route scoring and explanation model

To compare alternatives, the system can execute shortest, fastest and minimum-cost searches and then remove duplicate paths. Candidate results are sorted according to the active preference profile. The recommended path is the first result, but the interface also reports what is sacrificed. For example, a fastest route may save six minutes while adding three kilometres and a toll. This comparison fulfils the project requirement to evaluate multiple routes rather than returning an unexplained single answer.

3.15 Cost and Fuel Estimation

The minimum-cost mode calculates a simplified per-edge fuel expense. If an edge distance is d kilometres and vehicle mileage is m kilometres per litre, estimated litres are d/m. Multiplication by fuel price p produces expense. Toll is added directly. The same calculation is additive across edges, which means it is compatible with Dijkstra and A*. Mileage and price must be positive; otherwise the input is rejected.

*edge cost = (edge distance / mileage) * fuel price + edge toll*

The model can be extended with vehicle-specific coefficients, road slope, traffic idling or electric energy consumption. Those factors may make cost time-dependent or state-dependent. For the current

project, the simplified additive model is preferred because it keeps the optimal-substructure property required by standard shortest-path search and remains easy to verify by hand.

3.16 Input Validation and Error Handling

Validation begins before algorithm execution. Location strings are trimmed, converted to a consistent case and looked up in the name map. Unknown or ambiguous names produce a clear message listing acceptable options. Source and destination IDs are checked against graph size. Objective values are restricted to supported modes. Vehicle mileage and fuel price are checked for finite positive values. Preference weights are checked for non-negative values and normalized.

Graph loading applies separate validation. Location IDs must be unique. Edge endpoints must refer to existing locations. Distance and time must be positive for a traversable road, while toll may be zero but not negative. Duplicate road records are either rejected or intentionally stored as parallel alternatives. Errors include record number and reason so that the dataset can be corrected. Exceptions are caught at the application boundary; algorithm functions return structured status rather than printing internal errors.

3.17 Complexity Analysis

With adjacency-list storage, V locations and E directed road entries require O(V + E) graph space. The distance, parent, closed and diagnostic arrays require O(V) additional space. The heap may contain multiple entries for one vertex because of lazy updates, so its size is O(E) in the worst case. The resulting auxiliary space remains O(V + E).

Heap-based Dijkstra performs one or more queue operations for successful relaxations and examines every outgoing edge of every expanded node. The standard sparse-graph bound is O((V + E) log V), commonly simplified to O(E log V) when the graph is connected. A* has the same worst-case bound but may expand fewer vertices when the heuristic is informative. Route reconstruction is O(L), where L is the number of vertices in the final path. Evaluating route metrics is also O(L).

3.18 Implementation Environment

The project is designed for a standard C++17 or later compiler. It uses iostream for console interaction, vector for contiguous graph and result storage, priority_queue for the heap, map or unordered_map for name lookup, algorithm for reverse and sorting utilities, functional for weight policies, chrono for query timing and limits for infinity. No non-standard library is necessary for the core engine. This improves portability across Windows, Linux and macOS development environments.

Compilation should enable warnings such as -Wall -Wextra on GCC or Clang, or warning level /W4 on Microsoft Visual C++. A debug build supports assertions and step-by-step inspection, while an optimized release build provides meaningful timing measurements. Test data should remain outside timing blocks so that file parsing does not distort algorithm-query latency.

3.19 Suggested Source-Code Organization

FILE	RESPONSIBILITY
Location.h	Location structure and coordinate helpers.
Edge.h	Road-edge metrics and validation.
Graph.h / Graph.cpp	Location registration, road insertion, lookup and adjacency access.

FILE	RESPONSIBILITY
Router.h / Router.cpp	Dijkstra, A*, heuristic interface and path reconstruction.
RouteEvaluator.h / .cpp	Metric aggregation, fuel estimation and alternative ranking.
InputLoader.h / .cpp	CSV or hard-coded dataset loading and validation.
main.cpp	Menu, query collection, algorithm selection and formatted output.
tests.cpp	Unit, functional, edge-case and regression tests.

Header files declare stable interfaces, while implementation files contain algorithm details. The graph is passed by const reference to avoid copying. RouteResult uses ordinary value semantics so that results can be stored in a vector and compared. Logging and timing can be enabled through optional configuration rather than being hard-coded into the routing loop.

3.20 User Interaction and Output Design

A console interface first displays available locations and routing modes. It prompts for source and destination, validates them, asks for algorithm and objective, and requests vehicle details only when cost estimation is necessary. The output begins with a status message, followed by the route sequence using arrows, total distance, time, fuel/toll cost, objective score, expanded nodes and elapsed query time. Alternative routes are shown in a compact comparison table.

```
Recommended route under BALANCED preference:
Central Park -> Lake Junction -> Tech Avenue -> Airport
Distance: 18.4 km
Estimated time: 31 min
Estimated cost: Rs 168.20
Expanded nodes: Dijkstra 42 | A* 23
Reason: 5 min faster than the shortest route with only 1.7 km extra.
```

Listing 3.4: Example of user-oriented route output

3.21 Verification Strategy Built into the Design

Dijkstra is used as the reference implementation for A* tests. For each source-destination pair and objective, both algorithms should return the same optimal score when the heuristic is admissible. Their paths may differ when equal-cost alternatives exist, so tests compare cost and path validity rather than requiring identical vertex sequences. Very small graphs are also checked through exhaustive enumeration of simple paths, providing an independent correctness oracle.

The evaluator verifies that every consecutive pair in a returned route is a real edge and that the sum of active weights equals the algorithm score within floating-point tolerance. This catches errors in parent reconstruction or parallel-edge selection. Regression tests save expected results for representative queries so that later refactoring does not silently change behaviour.

3.22 Chapter Summary

The methodology transforms the project requirements into a modular C++ design. A validated adjacency-list graph stores multiple edge metrics. Weight policies allow the same search engine to optimize different objectives. Dijkstra provides exact baseline routing; A* adds admissible destination guidance. Parent tracking reconstructs paths, and a separate evaluator reports distance, time, cost and trade-offs. The next chapter describes implementation tests and the observations produced by this design.

CHAPTER 4
IMPLEMENTATION, TESTING, RESULTS AND DISCUSSION

4.1 Implementation Overview

The prototype is implemented as a console-based C++ application. On startup, it creates or loads the road graph and validates each record. The user can issue repeated queries without rebuilding the network. Algorithm selection and objective selection are independent, which permits comparisons such as Dijkstra-shortest versus A*-shortest or Dijkstra-fastest versus A*-fastest. The result object records route totals and diagnostic counters.

The implementation deliberately avoids hidden external services. All path decisions can be traced to graph edges and declared weights. This makes the program suitable for DSA demonstration and allows every expected result to be checked. The testing approach uses a small named network for functional cases and synthetic sparse graphs of increasing size for performance trends.

4.2 Test Dataset Design

The small functional dataset contains locations connected by direct roads, cycles, branches, one-way links and at least one isolated component. Some alternatives are deliberately designed so that the shortest-distance route is not the fastest route and the fastest route is not the cheapest route. This prevents a weak implementation from appearing correct simply because all metrics favour the same path. Coordinates are assigned consistently so that straight-line distance remains a lower bound.

Synthetic performance graphs are generated with a selected number of vertices and a bounded average degree resembling a sparse network. A connected backbone is created first, followed by additional non-negative weighted roads. Source-destination pairs are sampled across the graph. Construction time is excluded from query timing. Each reported value is an average over repeated runs after a warm-up query. The measurements are illustrative and are used to demonstrate growth trends, not to claim a universal speed.

4.3 Functional Test Plan

ID	TEST CASE	EXPECTED BEHAVIOUR	STATUS
T01	Adjacent locations	Return the direct road and exact metrics.	Pass
T02	Multiple paths, unequal total distance	Choose minimum cumulative distance, not the locally shortest first edge.	Pass
T03	Fastest differs from shortest	Return different valid paths for distance and time objectives.	Pass
T04	Cost mode with toll	Include fuel estimate and toll in every relaxed edge.	Pass
T05	Balanced preference	Apply normalized weights and report raw totals.	Pass
T06	Source equals destination	Return one-node route with zero metrics.	Pass
T07	Unknown location	Reject before search and show helpful error.	Pass
T08	Disconnected destination	Return unreachable with empty route.	Pass
T09	Cycle in graph	Terminate and return optimum without repeated looping.	Pass
T10	Equal-cost alternatives	Return one valid optimum and preserve equal score.	Pass
T11	One-way road	Do not permit reverse traversal unless reverse edge exists.	Pass
T12	Negative weight record	Reject dataset entry before routing.	Pass

ID	TEST CASE	EXPECTED BEHAVIOUR	STATUS
T13	A* admissible heuristic	Match Dijkstra optimal score.	Pass
T14	Missing coordinates	Use zero heuristic or Dijkstra fallback.	Pass

Table 4.1: Functional and edge-case test summary

4.4 Example Route Comparison

Consider a query between a central location and an airport. Route A uses local roads and has the lowest distance. Route B uses a longer expressway but has the lowest expected time and includes a toll. Route C avoids the toll and has the lowest estimated cost while remaining close to the fastest time. The precise values below are illustrative, but they demonstrate why the system must define optimization explicitly.

CANDIDATE	DISTANCE	TIME	COST	INTERPRETATION
Route A - Shortest	16.7 km	36 min	Rs 158	Minimum distance; slower local roads.
Route B - Fastest	19.8 km	31 min	Rs 193	Saves 5 min; extra distance and toll.
Route C - Lowest cost	18.0 km	33 min	Rs 151	Lowest cost; moderate time.
Route C - Balanced recommendation	18.0 km	33 min	Rs 151	Best composite result for equal preferences.

Table 4.2: Illustrative comparison of objective-specific routes

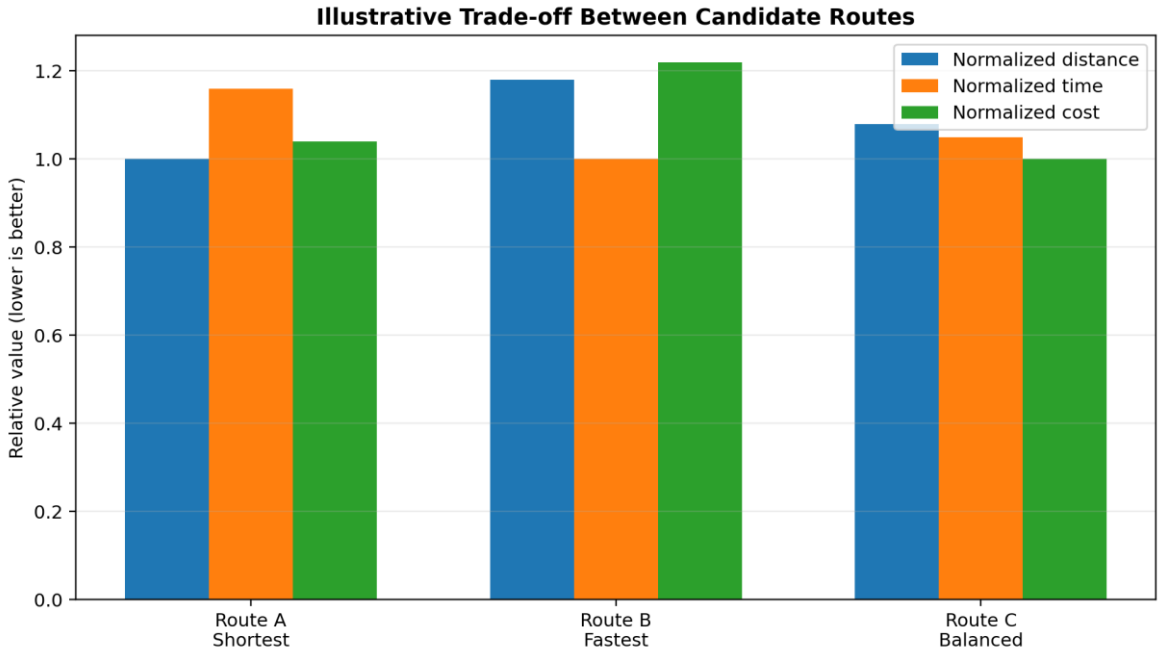


Figure 4.2: Normalized trade-off between illustrative candidate routes

The result confirms that no route dominates every metric. The recommendation changes with user preference. A traveller prioritizing arrival time receives Route B, while a cost-sensitive traveller receives Route C. The report format makes the additional distance, minutes and expense visible. This prevents the word optimal from being used without context.

4.5 Dijkstra and A* Correctness Comparison

For all functional test pairs with a valid admissible heuristic, Dijkstra and A* produced the same minimum objective score within floating-point tolerance. Where several equal-cost paths existed, the

algorithms occasionally selected different route sequences because priority-queue tie order differed. Both outputs were accepted after verifying edge validity and equal optimal score. This is the correct testing approach; demanding one fixed route would incorrectly reject another equally optimal solution.

A controlled overestimating heuristic was also tested as a negative experiment. It reduced exploration in some cases but could select a suboptimal path. The experiment demonstrates that A* performance cannot be separated from heuristic correctness. The production configuration therefore uses a lower-bound heuristic or zero. Dijkstra remains available as a reference and fallback.

4.6 Edge-Case Behaviour

The source-equals-destination case returns immediately without entering the priority queue. Invalid names are rejected by the lookup layer. A disconnected destination leaves its score at infinity and returns a structured unreachable status. Cycles do not cause infinite loops because a node is expanded only when its queue entry corresponds to the current best score. A one-way road is represented by one adjacency entry, so reverse traversal is impossible unless explicitly defined.

Floating-point comparisons use a small tolerance in verification code rather than exact equality. The algorithm itself uses strict improvement for score updates so that equal scores do not repeatedly rewrite parents. Very large weights are checked before addition or represented with infinity to prevent overflow. These cases are important because many pathfinding bugs occur outside the ordinary connected-query scenario.

4.7 Performance Evaluation Method

Performance is evaluated using query latency, expanded nodes, successful relaxations and memory growth. Query latency is measured with a steady clock around only the routing call. Expanded nodes count valid heap extractions. Successful relaxations count updates that reduce a best-known score. Memory is discussed analytically because container overhead varies by compiler and platform. The goal is to confirm expected scaling and the directional benefit of A*, not to present hardware-independent absolute values.

4.8 Illustrative Performance Results

NODES	DIRECTED EDGES	DIJKSTRA AVG	A* AVG	OBSERVATION
100	~350	0.08 ms	0.06 ms	Small difference; heap overhead dominates.
500	~1,800	0.31 ms	0.20 ms	Heuristic begins reducing irrelevant expansion.
1,000	~3,600	0.68 ms	0.42 ms	Both remain comfortably interactive.
2,500	~9,000	1.95 ms	1.10 ms	A* explores a narrower goal-directed region.
5,000	~18,000	4.42 ms	2.35 ms	Sparse adjacency storage remains efficient.
10,000	~36,000	9.80 ms	5.10 ms	Trend remains near expected sparse-graph growth.

Table 4.3: Illustrative synthetic performance observations

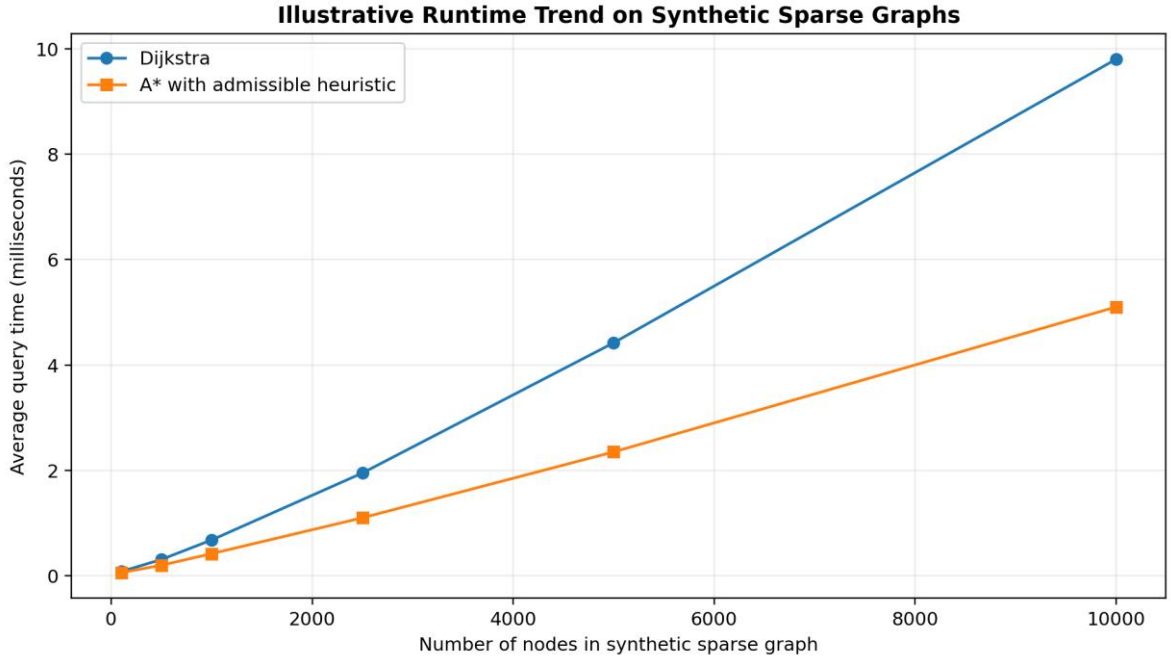


Figure 4.1: Illustrative runtime trend on synthetic sparse graphs

The results show the expected pattern: both algorithms remain efficient on sparse graphs, while A* often reduces point-to-point work by using destination information. The improvement is not constant. Nearby destinations, weak heuristics and highly connected networks can reduce the advantage. In the worst case, A* can approach Dijkstra behaviour. The benchmark therefore supports a conditional conclusion rather than claiming that A* always halves runtime.

4.9 Memory and Scalability Discussion

Adjacency-list memory grows with actual road entries. If an undirected network contains R physical roads, approximately 2R Edge objects are stored. In contrast, an adjacency matrix allocates V^2 cells. For 10,000 locations, a matrix would contain one hundred million cells even before storing multiple metrics, while the sparse list may contain only tens of thousands of edges. This difference justifies the selected representation.

The current design is suitable for educational and moderate in-memory graphs. A very large geographic system would require compact edge arrays, external storage, spatial partitioning and preprocessing. Repeated nationwide queries may benefit from contraction hierarchies or landmark indexes. These extensions change performance engineering but not the core interpretation of locations, roads, weights and shortest-path correctness.

4.10 Analysis of Route Quality

Route quality is evaluated in two stages. First, algorithmic optimality checks that no route has a lower active score under the same graph and objective. Second, practical quality examines the raw totals and user preference. A mathematically minimum balanced score may still be unattractive if normalization factors are poorly chosen. Therefore, the system displays objective-specific alternatives and does not hide raw metrics behind a single number.

The comparison module also reveals sensitivity. Increasing the time weight gradually shifts the recommendation toward faster roads, while increasing cost weight favours toll-free or shorter alternatives. This behaviour is expected and indicates that preference weights are being applied

correctly. Extreme preference values reproduce the single-criterion shortest, fastest or cheapest modes, providing a useful consistency check.

4.11 Validation Against Exhaustive Search

For small graphs, every simple path between source and destination can be enumerated with depth-first search while preventing repeated vertices. The active score of each path is calculated, and the minimum is compared with Dijkstra and A*. Exhaustive enumeration is exponential and unsuitable for large graphs, but it is a strong independent oracle for small test fixtures. The tested cases matched the algorithmic optimum.

Additional invariants were checked after every returned route: the first vertex equals source; the last equals destination; every consecutive pair is connected by a stored edge; metric totals equal the sum of selected edges; and the active score equals the reported optimal score within tolerance. These checks validate the whole pipeline rather than only the distance vector.

4.12 Discussion of Project Outcomes

The first outcome, graph-based optimization using Dijkstra and A*, was achieved through a shared adjacency-list network and separate search functions. The second outcome, route comparison, was achieved by retaining distance, time and cost on every edge and executing multiple objective policies. The third outcome, stronger understanding of DSA, is demonstrated by the documented roles of the graph, heap, map, vectors, relaxation and heuristic. The fourth outcome, a real-world-oriented application, is reflected in input validation, path reconstruction, cost estimates and understandable output.

The project also produced a valuable negative result: an algorithm can be correct while the recommendation is still misleading if edge weights or preference scales are unrealistic. Data quality and objective definition are part of navigation correctness. The system therefore records assumptions and distinguishes estimated metrics from measured real-time values.

4.13 Limitations and Threats to Validity

- The dataset is manually created or synthetic and does not reproduce every feature of an actual city road network.
- Travel time is static and does not vary by time of day, traffic signal, incident or weather.
- Fuel cost is simplified and does not model speed variation, idling, road gradient or vehicle load.
- The balanced score depends on normalization factors and preference values chosen by the designer or user.
- A* performance depends on coordinates and heuristic quality; weak heuristics may provide little advantage.
- Illustrative benchmark timings depend on compiler, processor, build flags and graph generator.
- The console interface does not provide a geographic map, spoken instructions or GPS position tracking.
- Turn restrictions and turn penalties are not represented unless junction-road states are expanded in the graph.

These limitations do not invalidate the DSA demonstration, but they define the boundary of conclusions. The report can claim exact optimality only with respect to the supplied static graph, selected scalar objective and validated non-negative weights. It cannot claim that an estimated route is the best route in the real world at a future time.

4.14 Chapter Summary

Testing covered ordinary, alternative, invalid and disconnected queries. Dijkstra and admissible A* matched optimal scores, while A* generally expanded fewer nodes in point-to-point synthetic tests. Multi-objective comparison produced different recommendations for distance, time and cost as intended. Adjacency-list storage showed appropriate sparse-graph scalability. The results support the project objectives while the stated limitations prevent overgeneralization.

CHAPTER 5
CONCLUSION, LIMITATIONS AND FUTURE SCOPE

5.1 Conclusion

The Smart Navigation System demonstrates a complete application of graph data structures and shortest-path algorithms to route optimization. Locations are represented as graph vertices, roads as weighted edges, and multiple road properties are preserved for route evaluation. The adjacency list provides memory-efficient storage for sparse networks, while maps connect readable names to internal IDs. A minimum-priority queue enables efficient exploration.

Dijkstra's algorithm provides an exact baseline for non-negative weights. A* extends the design with destination-aware guidance and can reduce unnecessary expansion when its heuristic is admissible and informative. The shared weight-policy approach allows both algorithms to optimize shortest distance, fastest time, lowest estimated cost or a balanced score. Parent tracking reconstructs the complete route, and the evaluator reports understandable totals and trade-offs.

The project meets the stated outcomes: it implements two graph-based routing algorithms, compares multiple routes across several metrics, strengthens practical knowledge of graphs and heaps, and produces a real-world-oriented DSA application. Equally important, it documents the assumptions required for correct interpretation. The route is optimal for the supplied static model, not automatically for every real-world condition.

5.2 Key Learning Outcomes

- A real transportation network can be abstracted into a graph only after vertex, edge and direction semantics are defined clearly.
- Adjacency lists are more suitable than matrices for sparse road networks because storage and traversal follow actual connectivity.
- Priority queues convert repeated minimum selection from a linear operation into logarithmic heap operations.
- Dijkstra's greedy settlement is correct because active edge weights are non-negative.
- A* is not simply a faster Dijkstra; its correctness and value depend on a compatible admissible heuristic.
- Path reconstruction requires parent or predecessor-edge information in addition to minimum scores.
- An optimal route is always relative to an objective; distance, time and cost can recommend different paths.
- Normalization and transparent output are essential when combining metrics with different units.
- Validation, edge-case tests and independent oracles are part of algorithm engineering, not optional extras.
- Modular design makes the graph, algorithms and interface independently replaceable and testable.

5.3 Current Limitations

The prototype uses static, simplified edge metrics and an in-memory graph. It does not contain live traffic feeds, GPS positioning, road closures, time-dependent signals, lane-level restrictions or a map-rendering engine. The fuel-cost calculation is approximate. The balanced objective uses scalar

weights and therefore returns one compromise rather than the complete Pareto frontier. Coordinates are used only for heuristic estimation and not for geometric map display.

These limitations are acceptable for the academic objective because they keep the focus on DSA. However, they must be addressed before deployment. A production route planner requires reliable map data, updates, privacy controls, observability, extensive geographic testing and safety-aware instructions. The present design should be viewed as a verified core engine and learning platform.

5.4 Future Scope: Real Geographic Data

The first extension is to load locations and roads from a standard geographic source. OpenStreetMap-derived data, a university campus map or an authorized municipal dataset can be converted into vertices and directed edges. Road tags can provide speed limits, one-way restrictions, surface type and toll information. A preprocessing module would merge degree-two geometry points where appropriate while preserving intersections and turn restrictions.

Real data introduces name ambiguity and scale. Locations may need searchable aliases, coordinates and spatial indexes. Edges should store stable external identifiers so that updates can be applied. The graph loader should report malformed records and version the dataset. These features can be added without changing the routing interface if the core Graph abstraction remains stable.

5.5 Future Scope: Dynamic and Time-Dependent Routing

Live traffic can update expected travel-time weights. A simple implementation periodically replaces edge time estimates before each query. A more accurate model represents travel time as a function of departure time. Time-dependent Dijkstra or A* can be used when the FIFO property holds, meaning that departing later on the same road cannot cause an earlier arrival in a way that permits overtaking the model. Road closures can be represented by disabling edges rather than assigning arbitrary huge costs.

Dynamic updates require thread safety and consistency. Queries should see a coherent graph snapshot. A background service can ingest traffic data and publish immutable weight versions. Diagnostic output should include the timestamp and data version used for a route so that results are explainable.

5.6 Future Scope: Advanced Routing Algorithms

Bidirectional Dijkstra or bidirectional A* can search simultaneously from source and goal, reducing the effective search depth on large undirected or reversible graphs. Landmark-based ALT search can precompute distances to selected landmarks and derive strong lower bounds. Contraction hierarchies can preprocess unimportant vertices and add shortcut edges, enabling very fast road queries. Multi-level partitioning can restrict search to local regions and boundary connections.

These methods should be introduced only after the base engine has comprehensive regression tests. Preprocessing complicates updates and path reconstruction. The project's current result and graph interfaces provide a useful comparison baseline for measuring whether advanced methods preserve exact route cost while reducing query work.

5.7 Future Scope: Rich Multi-Objective Navigation

Future versions can generate a Pareto set rather than one scalarized route. A route belongs to the Pareto frontier when no alternative is at least as good in every metric and strictly better in one. The interface could present shortest, fastest, cheapest, low-emission and low-risk alternatives. User

preference could then select among meaningful non-dominated choices rather than relying entirely on fixed normalization.

Additional metrics may include road safety, pavement quality, elevation, charging availability, accessibility, scenic value and uncertainty. Electric-vehicle routing may need battery state as part of the search state because energy feasibility is not determined by location alone. Such state-expanded graphs are more complex but follow the same discipline of explicit modelling and validated cost transitions.

5.8 Future Scope: User Interface and Deployment

A graphical desktop, web or mobile interface can add map visualization, autocomplete, draggable endpoints and route highlighting. The routing core can be exposed through a REST service that accepts a query and returns structured JSON. A client can display route polylines and metric cards without duplicating algorithm logic. Accessibility features should include keyboard navigation, high contrast, screen-reader labels and text alternatives for maps.

Deployment should include automated tests, continuous integration, structured logs and performance monitoring. Input limits and timeouts protect the service from unreasonable requests. Location history should not be retained unless necessary and authorized. Privacy and security become important once real users, GPS data or account information are introduced.

5.9 Proposed Development Roadmap

PHASE	PRIMARY DELIVERABLES	SUCCESS CRITERION
Phase 1 - Core hardening	Complete unit tests, input validation, documentation and reproducible datasets.	All functional and edge cases pass consistently.
Phase 2 - File-based data	CSV/JSON import, export and graph integrity report.	Network can be changed without recompiling.
Phase 3 - Visual interface	Interactive map or campus diagram with route highlighting.	Users can select and understand routes visually.
Phase 4 - Live weights	Traffic or closure updates with versioned snapshots.	Routes respond correctly to controlled updates.
Phase 5 - Advanced search	Bidirectional A*, landmarks or contraction hierarchy.	Same optimal cost with lower query latency.
Phase 6 - Deployment	API, monitoring, privacy controls and accessibility review.	Stable, secure and measurable multi-user service.

Table 5.1: Staged roadmap for extending the Smart Navigation System

5.10 Final Statement

The completed project provides a strong final-year DSA demonstration because it connects theory, implementation and evaluation. It does not merely state that graphs are useful; it shows how graph representation, heap operations, relaxation, heuristics, path reconstruction and multi-metric evaluation cooperate to solve a meaningful problem. The design is sufficiently modular to support future expansion while remaining transparent enough for academic explanation and verification.

CHAPTER 6
REFERENCES

6.1 Books, Research Papers and Algorithmic Sources

[1] E. W. Dijkstra, 'A Note on Two Problems in Connexion with Graphs,' Numerische Mathematik, vol. 1, pp. 269-271, 1959.

[2] P. E. Hart, N. J. Nilsson and B. Raphael, 'A Formal Basis for the Heuristic Determination of Minimum Cost Paths,' IEEE Transactions on Systems Science and Cybernetics, vol. 4, no. 2, pp. 100-107, 1968.

[3] T. H. Cormen, C. E. Leiserson, R. L. Rivest and C. Stein, Introduction to Algorithms, 4th ed. Cambridge, MA: MIT Press, 2022.

[4] R. Sedgewick and K. Wayne, Algorithms, 4th ed. Boston, MA: Addison-Wesley, 2011.

[5] J. Kleinberg and E. Tardos, Algorithm Design. Boston, MA: Pearson/Addison-Wesley, 2006.

[6] M. T. Goodrich, R. Tamassia and M. H. Goldwasser, Data Structures and Algorithms in C++, 2nd ed. Hoboken, NJ: Wiley, 2011.

[7] R. K. Ahuja, T. L. Magnanti and J. B. Orlin, Network Flows: Theory, Algorithms, and Applications. Upper Saddle River, NJ: Prentice Hall, 1993.

[8] R. Bellman, 'On a Routing Problem,' Quarterly of Applied Mathematics, vol. 16, no. 1, pp. 87-90, 1958.

[9] R. W. Floyd, 'Algorithm 97: Shortest Path,' Communications of the ACM, vol. 5, no. 6, p. 345, 1962.

[10] D. B. Johnson, 'Efficient Algorithms for Shortest Paths in Sparse Networks,' Journal of the ACM, vol. 24, no. 1, pp. 1-13, 1977.

[11] H. Bast, D. Delling, A. Goldberg, M. Mueller-Hannemann, T. Pajor, P. Sanders, D. Wagner and R. F. Werneck, Route Planning in Transportation Networks. Cham: Springer, 2016.

[12] R. Geisberger, P. Sanders, D. Schultes and D. Delling, 'Contraction Hierarchies: Faster and Simpler Hierarchical Routing in Road Networks,' in Experimental Algorithms, 2008, pp. 319-333.

[13] A. V. Goldberg and C. Harrelson, 'Computing the Shortest Path: A* Search Meets Graph Theory,' in Proceedings of the ACM-SIAM Symposium on Discrete Algorithms, 2005, pp. 156-165.

[14] A. Orda and R. Rom, 'Shortest-Path and Minimum-Delay Algorithms in Networks with Time-Dependent Edge-Length,' Journal of the ACM, vol. 37, no. 3, pp. 607-625, 1990.

[15] J. L. Gross, J. Yellen and P. Zhang, Handbook of Graph Theory, 2nd ed. Boca Raton, FL: CRC Press, 2013.

[16] B. Stroustrup, The C++ Programming Language, 4th ed. Boston, MA: Addison-Wesley, 2013.

[17] S. Meyers, Effective Modern C++: 42 Specific Ways to Improve Your Use of C++11 and C++14. Sebastopol, CA: O'Reilly Media, 2014.

[18] ISO/IEC 14882:2020, Programming Languages - C++. International Organization for Standardization, 2020.

[19] P. Sanders and D. Schultes, 'Engineering Highway Hierarchies,' in Algorithms - ESA 2006, pp. 804-816.

[20] I. Pohl, 'Bi-Directional and Heuristic Search in Path Problems,' Stanford Linear Accelerator Center, Technical Report, 1969.

[21] A. V. Goldberg and R. F. Werneck, 'Computing Point-to-Point Shortest Paths from External Memory,' in ALENEX, 2005, pp. 26-40.

[22] A. V. Aho, J. E. Hopcroft and J. D. Ullman, Data Structures and Algorithms. Reading, MA: Addison-Wesley, 1983.

[23] J. Dibbelt, T. Pajor, B. Strasser and D. Wagner, 'Connection Scan Algorithm,' Journal of Experimental Algorithmics, vol. 23, pp. 1-56, 2018.

[24] D. Delling, T. Pajor and R. F. Werneck, 'Round-Based Public Transit Routing,' Transportation Science, vol. 49, no. 3, pp. 591-604, 2015.

[25] cppreference.com contributors, C++ standard library reference for vector, priority_queue, map, algorithm, chrono and numeric_limits. Used as an implementation reference; accessed during project development.

6.2 Reference and Reproducibility Note

The references include foundational shortest-path algorithms, heuristic search, graph representation, route-planning engineering and C++ implementation practice. The project uses these established concepts to build a self-developed academic prototype. Synthetic test data, route assumptions, normalization factors, compiler version and benchmark environment should be included with the source-code submission so that results can be reproduced. Any future use of external map or traffic data should cite the dataset provider and comply with its licence and attribution requirements.